MNIST Digit Recognizer on FPGA: Spec

Oct 9, 2026 · @Evan

Overview

The project builds an MNIST digit recognizer in SystemVerilog in two phases: a fully connected network (v1), then a small CNN (v2) on the same hardware framework, so v2 adds blocks instead of replacing them.

Phase	Network	Hardware added	Done when
v1: MLP	784 → 64 → 10	Full framework: I/O, datapath, buffers, controller, argmax	Bit-exact with the golden model on the test set
v2: CNN	3×3 conv (8 filters) → 2×2 max-pool → dense 1,352 → 10	Convolution address generator, max-pool unit	Same, with v2 weights

v1 is a complete, resume-ready project on its own; v2 is a planned upgrade.

Task and data

Classify 28×28 MNIST digits into 10 classes using binarized pixels.

Item	Choice
Dataset	MNIST: 60,000 training and 10,000 test images
Input	28 × 28 = 784 pixels, grayscale 0–255
Binarization	Pixel ≥ 128 → 1, otherwise 0; the same rule in training, golden model and hardware
Classes	10 (digits 0–9)
Prediction	Index of the largest output score
Packed size	98 bytes per image (8 pixels per byte)

Network architecture

Both networks use ReLU on hidden layers and raw scores at the output; v2 replaces v1's hidden dense layer with a convolution and pooling stage but keeps a dense output layer.

v1: MLP

Layer	Output shape	Weights	Biases	MACs per image
Dense + ReLU	64	50,176	64	50,176
Dense (scores)	10	640	10	640
Total		**50,816**	**74**	**50,816**

v2: CNN

Layer	Output shape	Weights	Biases	MACs per image
Conv 3×3, 8 filters, no padding, + ReLU	26 × 26 × 8 = 5,408	72	8	48,672
Max-pool 2×2, stride 2	13 × 13 × 8 = 1,352	0	0	0 (comparisons only)
Dense (scores)	10	13,520	10	13,520
Total		**13,592**	**18**	**62,192**

- **Flatten order:** channel, then row, then column, matching PyTorch's flatten; the hardware buffer layout uses the same order.
- **No softmax:** it never changes which score is largest.
- **Expected accuracy** (approximate, to confirm in training): v1 about 97%, v2 about 98%.

Number formats and quantization

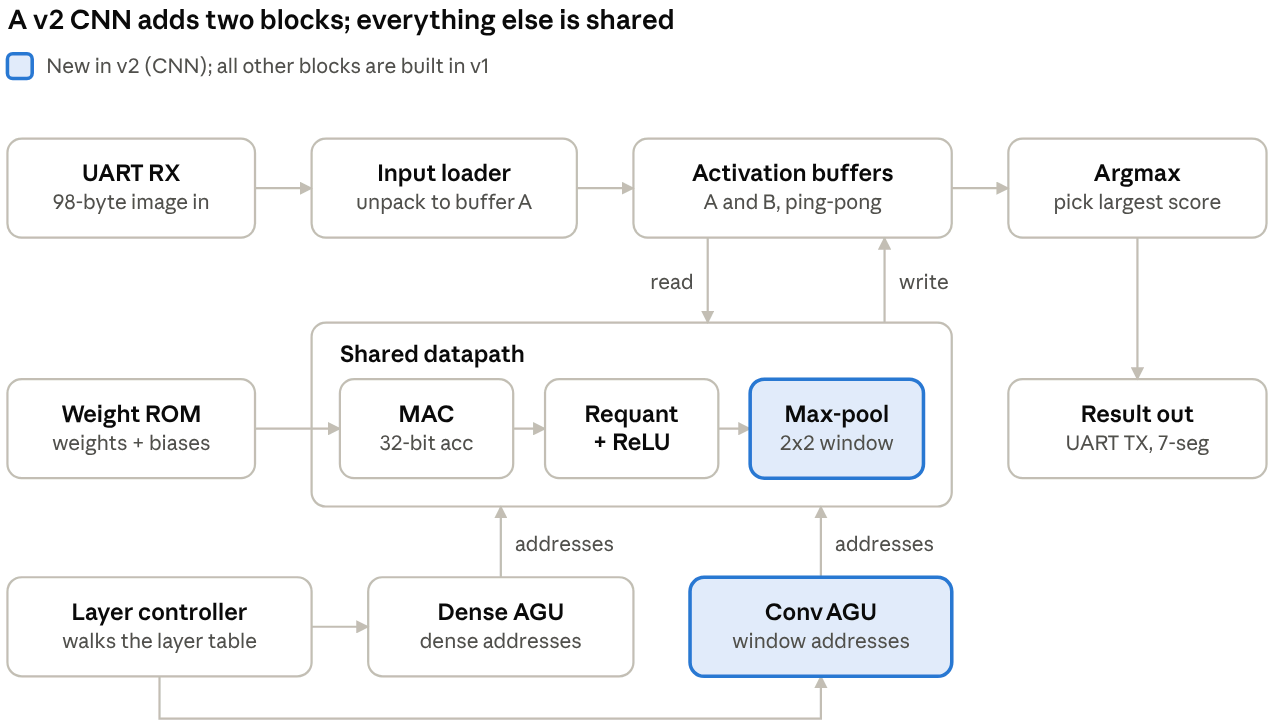
Every layer type uses the same integer formats and the same requantize step, so convolution and pooling layers drop into v1's datapath unchanged.

Signal	Format	Notes
Input pixels	0 or 1, stored as a normal activation	The first layer reads them like any other layer
Weights	8-bit signed fixed point	Fractional bits chosen per layer from the trained weight ranges
Biases	32-bit signed, pre-scaled to the accumulator format	Loaded as the accumulator's starting value
Accumulator	32-bit signed	The golden model checks that no layer's worst case exceeds it
Activations	16-bit signed	Stored in the activation buffers
Requantize	Arithmetic right shift by a per-layer amount, saturate to 16 bits, then ReLU if the layer has it	One block shared by all layer types

Per-layer shift amounts and fractional bits are set in the golden model and exported alongside the weights.

Hardware architecture

Every layer is one pass of a shared datapath from one activation buffer to the other, so the layer types v2 needs arrive as two new blocks, not a new design.



proposed hardware architecture · v1 blocks plus two v2 additions

Images flow left to right along the top row; the layer controller drives the address generators, which steer the shared datapath through one layer at a time.

Design rules that keep the upgrade cheap:

- **The datapath knows nothing about layers.** The MAC only receives a weight, an activation, and clear/accumulate signals.
- **Layers are data, not code.** A small layer table stores each layer's type, sizes, weight and bias base addresses, shift amount and ReLU flag; the controller steps through it. Adding a layer means adding a table entry.
- **Ping-pong buffers.** Each layer reads buffer A or B and writes the other.
- **Sizes are parameters**, never hard-coded.
- **Every block has a start/done handshake**, so each can be tested on its own.

Module	Job	Phase
UART RX / TX	Serial I/O with the PC	v1
Input loader	Unpacks the 98-byte image into buffer A	v1
Layer controller	Steps through the layer table	v1

Module	Job	Phase
Dense AGU	Generates addresses for dense layers	v1
Conv AGU	Generates addresses for 3×3 windows	v2
MAC	Multiply-accumulate with a 32-bit accumulator	v1
Requant + ReLU	Shift, saturate, ReLU	v1
Max-pool	Max of each 2×2 window	v2
Weight ROM	Weights and biases, loaded with `$readmemh`	v1
Activation buffers	A and B, ping-pong	v1
Argmax	Picks the largest of the 10 scores	v1

AGU = address generation unit.

Memory budget

Both phases fit easily in the Basys 3's roughly 225 KB of block RAM, and the activation buffers are sized for v2 from the start so the upgrade needs no resizing.

Memory	v1	v2
Weights (8-bit)	50,816 B ≈ 50 KB	13,592 B ≈ 13 KB
Biases (32-bit)	296 B	72 B
Activation buffers A + B (5,408 × 16-bit each)	21,632 B ≈ 21 KB	21,632 B ≈ 21 KB
Total	**≈ 73 KB**	**≈ 35 KB**

v2 needs less memory than v1 because convolution reuses its 72 filter weights across the whole image.

I/O

Images arrive over the board's USB-UART, and the predicted digit goes back to the PC and onto the seven-segment display.

Direction	Bytes	Contents
PC → FPGA	1	Start byte `0xAA`
PC → FPGA	98	Image, 8 pixels per byte, row by row
FPGA → PC	1	Predicted digit, 0–9
FPGA → PC (optional debug)	20	The 10 raw scores as 16-bit values

- **UART:** 115,200 baud, 8 data bits, no parity, 1 stop bit, about 868 clock cycles per bit at 100 MHz.
- **Speed:** sending an image takes about 8.6 ms, while inference takes about 0.5 ms (v1) or 0.6 ms (v2) with one MAC at 100 MHz, so the UART sets the throughput.
- **On the board:** the seven-segment display shows the digit, and the LEDs show the controller state for debugging.

Verification and success criteria

The hardware must match a bit-accurate Python golden model exactly, layer by layer, on every test image.

- **Golden model:** an integer-only Python version of each layer type (dense, convolution, max-pool, requantize) that exports every layer's output for each test image.
- **Unit tests:** each module passes its own self-checking testbench before integration.
- **Layer tests:** each layer's buffer contents are compared with the golden model's export.
- **Full test:** 1,000 images in simulation during development, then all 10,000, using Verilator for speed.

Criterion	v1 target	v2 target
Float model accuracy	≥ 97%	≥ 98%
Quantized golden model	Within 1% of float	Within 1% of float
Hardware vs. golden model	100% exact match	100% exact match
Timing on Basys 3	Meets 100 MHz	Meets 100 MHz

Open decisions

- [ ] Bit order of pixels within each UART byte
- [ ] Weight fractional bits and shift amount for each layer, from the trained weight ranges
- [ ] Rounding when requantizing: truncate or round to nearest
- [ ] Test framework: SystemVerilog testbenches or cocotb
- [ ] Whether v2 adds a second dense layer before the output
- [ ] When to add parallel MAC lanes (after v1 works on the board)

Build checklist

Tick each item when it passes its own test, and commit it to GitHub at the same time.

Setup

- [ ] Create the GitHub repo with a README, a `.gitignore` and a license
- [ ] Create the folder structure (see GitHub workflow below)
- [ ] Install Python with PyTorch, Icarus Verilog or Verilator, and GTKWave
- [ ] Link this spec from the README

Phase 1: Python

- [ ] Load and binarize MNIST
- [ ] Train the 784-64-10 MLP and record float accuracy (target ≥ 97%)
- [ ] Count dead hidden neurons
- [ ] Choose fractional bits and shift amounts per layer
- [ ] Write the integer-only golden model and record quantized accuracy (within 1% of float)
- [ ] Export `weights.mem`, `biases.mem` and the layer shifts
- [ ] Export test vectors: images, per-layer outputs, expected digits

Phase 2: Building blocks

- [ ] `mac` + testbench
- [ ] `requant_relu` + testbench
- [ ] `act_buf` + testbench (one-cycle read latency)

- [ ] `rom` + testbench (`$readmemh`, negative values read correctly)
- [ ] `dense_agu` + testbench against a Python-generated address sequence
- [ ] `argmax` + testbench

Phase 3: Integration (simulation)

- [ ] One layer end to end matches the golden model's layer-1 output
- [ ] `layer_table` and `layer_ctrl`
- [ ] `mnist_core` with the image load port
- [ ] Core testbench: 10 images match the golden model
- [ ] 1,000 images match, then all 10,000
- [ ] Record accuracy and cycles per image

Phase 4: FPGA-readiness checks

- [ ] `verilator --lint-only` runs clean
- [ ] Vivado synthesis-only run: memories became block RAM, the multiply became a DSP block
- [ ] Save the resource and timing reports

Phase 5: Basys 3

- [ ] `uart_rx` and `uart_tx` + testbenches
- [ ] `input_loader`
- [ ] `seven_seg`
- [ ] Board top module and constraints file
- [ ] Python host script sends images and reads digits
- [ ] Record a demo video

Phase 6: Write-up

- [ ] README with block diagram, results table and lessons learned
- [ ] Update resume bullets with the real numbers

GitHub workflow

Keep the repo current by committing every finished checklist item and tagging each phase.

```
mnist-fpga/
├── README.md
├── docs/      spec, diagrams, results
├── python/    training, quantization, golden model
├── mem/       exported weights, biases, test vectors
├── rtl/       synthesizable design
├── tb/        testbenches
└── fpga/      board wrapper and constraints (Phase 5)
```

- **Commit per item:** one commit when a module passes its testbench, named for what it adds, like "Add mac with self-checking testbench".
- **Tag milestones:** `v0.1-python`, `v0.2-blocks`, `v1.0-sim`, `v1.1-fpga`.
- **Commit the small exports** in `mem/` so others can run the simulation without retraining; keep the raw MNIST download and build outputs out through `.gitignore`.
- **Keep a Status line** at the top of the README, updated at each milestone.
- **Optional:** a script or Makefile that runs every testbench with one command, and later a GitHub Actions workflow that runs them on every push.